INTERNAL ENGINEERING TRAINING

Vibe Coding, Done Properly

A practical team handbook for AI-assisted development — the mistakes to stop making, how to prompt without burning tokens, the rules and files every repo needs, and how we verify AI output instead of trusting it.

- Prompt playbook
- Tangible sprint phases
- Cursor & Antigravity rules
- Token economy
- Vitest & Playwright
- Cheat sheet

PRESENTER

Jagdish Harsh

VENUE

Mobiloitte HQ

SESSION

Friday, 22 May 2026

HOW TO USE THIS — PRESENTED BY JAGDISH HARSH

Contents & Reading Guide

A working handbook, not a hype piece. Vibe coding is a real productivity tool with real failure modes — this is about getting the upside while avoiding the downside. Each module ends with a take-home line. Keep the cheat sheet open while you work.

THE ONE IDEA BEHIND ALL ELEVEN MODULES

The AI drafts. You are still the engineer of record — every line in the repo is yours to review, understand, and defend.

MODULE 01 OF 11

01 What Vibe Coding Actually Is

Vibe coding is using AI coding agents — Cursor, Antigravity, and similar tools — to generate, edit, and run code from natural-language intent, with the developer steering and verifying at every step. The emphasis is on the second half.

It is a force multiplier, not autopilot

An AI agent is fast, tireless, and broadly knowledgeable. It is also **confidently wrong on a regular basis**. It will invent APIs that don't exist, miss edge cases, pick an outdated pattern, or quietly introduce a security hole — and present all of it with the same calm certainty as correct code. Confidence is not correctness. Treat every output as a draft from a fast junior developer who never says "I'm not sure."

The human-in-the-loop principle

The skill of vibe coding is not typing less — it is **specifying clearly, reviewing carefully, and verifying independently**. When you use an agent, the bottleneck shifts from writing code to reading it. If you cannot read and judge the diff, you cannot use the tool safely. "I don't fully understand this but it runs" is not done.

Where it genuinely shines

- Boilerplate, scaffolding, repetitive CRUD
- Refactors with a clear target shape
- Generating test scaffolding and fixtures
- Exploring and explaining unfamiliar code
- Debugging — generating hypotheses fast

Where to slow down and verify hard

- Auth, payments, anything security-sensitive
- Novel architecture decisions
- Code touching money, data deletion, or PII
- Anything you cannot personally verify
- Performance-critical or concurrency code

Set the expectation honestly

Vibe coding does not make a weak engineer strong. It makes a careful engineer faster and a careless engineer dangerous faster. The rest of this handbook is the "careful" part.

TAKE THIS HOME

The AI drafts. You sign off. Every line in the repo is yours to defend.

02

MODULE 02 OF 11

The Mistakes We're Making

These are the eight habits costing us time, tokens, and quality right now. None are exotic — they are the default when a team adopts AI tools without a shared standard. Each is fixable this week.

1

Jumping straight to "build me X" with no plan

Why it hurts: the agent guesses scope, structure, and intent. You discover the guess was wrong only after the code exists.

Do instead: ask for a plan first, review it, then implement. See Module 05.

2

Vague prompts → wrong output → re-prompt loop

Why it hurts: every retry re-sends context and re-reads files. Four vague attempts cost roughly four times the tokens of one precise prompt.

Do instead: spend 60 seconds on a tight prompt. See Modules 03 & 04.

3

Accepting code without reading it

Why it hurts: "looks fine, merge" ships bugs, dead code, and security holes that nobody owns or understands.

Do instead: read every line of every AI diff before it lands.

4

No rules files — re-explaining the stack every session

Why it hurts: wasted tokens, inconsistent output, and the agent drifting from our conventions in every new chat.

Do instead: set up `AGENTS.md` and scoped rules. See Module 07.

5

Secrets in code; `.env` committed to git

Why it hurts: once a key is in git history it is exposed forever — deleting the file does not remove it.

Do instead: `.gitignore` before first commit. See Module 09.

6

No tests — trusting "the AI said it works"

Why it hurts: AI code looks right and is confidently wrong. Without tests, the user finds the bug for you.

Do instead: every feature ships with tests that run in CI. See Module 10.

7

Giant multi-feature changes in one prompt

Why it hurts: the diff becomes unreviewable, untestable, and impossible to revert cleanly when one part breaks.

Do instead: one task per chat, one concern per PR. Small diffs.

8

Not documenting decisions — context lives only in chat

Why it hurts: chat history is private, unsearchable by the team, and disappears. The "why" behind decisions is lost.

Do instead: plans, sprint notes, and checkpoints go in markdown files. See Modules 05 & 06.

TAKE THIS HOME

Every item here is a habit — and habits change. Pick three. Fix them this sprint.

MODULE 03 OF 11

03 Prompting & the Token Economy

Tokens are billed input and output. A vague prompt does not just produce a bad answer — it produces a bad answer *and* a retry loop, and every retry re-pays for the context. Prompt quality is a direct cost line.

Why vague prompts are expensive

When a prompt is unclear, the agent guesses. The guess is wrong, so you re-prompt. Each re-prompt re-sends the conversation, re-reads the relevant files, and generates a fresh attempt. A task that should be one focused exchange becomes four or five — same result, several times the token cost and several times the wall-clock time.

Vague prompt — triggers the loop

```
"make the login better and fix
the bugs and add validation"
```

- No file named → agent searches blindly
- "Better" is undefined → agent guesses
- "The bugs" — which bugs?
- No acceptance criteria → no way to know it's done

Tight prompt — one clean exchange

```
"In src/auth/LoginForm.tsx, add
email-format + required-field
validation. Show errors inline
below each field. Don't change
the submit handler. Match the
Zod pattern in SignupForm.tsx."
```

The anatomy of a good prompt

Part	What to include
Goal	The one outcome you want, in a sentence.
Context	The exact file(s) and the existing pattern to follow or reuse.
Constraints	What must *not* change; libraries to use or avoid; style to match.
Acceptance	How both you and the agent know it's done — testable criteria.
Out of scope	Explicitly name what to leave alone, so the agent doesn't "improve" it.

Token hygiene — everyday habits

- **One task, one chat.** Don't drag a 200-message thread into a new task — all of it is re-sent every turn. Fresh chat when the topic changes.
- **Point at exact files.** Naming `src/auth/LoginForm.tsx` beats letting the agent search the whole repo.
- **Let rules files carry the constants.** Stack, conventions, and structure belong in `AGENTS.md` — not retyped every prompt (Module 07).
- **Don't paste what the agent can index.** Modern agents read the repo; pasting whole files just inflates the bill.
- **Scope small.** A narrow task is cheaper, faster, and far easier to review.

TAKE THIS HOME

A vague prompt is never free — you pay for it three times over. Sixty seconds of precision beats four dead retries.

MODULE 04 OF 11

04 The Prompt Playbook

Reusable prompt templates for tasks we do constantly. Copy them, adapt the `[bracketed]` parts, and keep them in a shared team snippet file. A prompt you reuse is a prompt that has already been debugged.

Prompt 1 — Set up a component-wise structure

An agent's default is to cram a whole feature into one enormous file. Before any code is written, make it propose a clean, modular structure — and review that structure the same way you'd review code.

▸ COPY-PASTE PROMPT

```
We're adding the [Lead Inbox] feature to a [Next.js + TypeScript] app.

Before writing any code, propose a component-wise file structure:
- List every component, its single responsibility, and its file path
- Separate presentational components from data/container components
- Identify shared components vs feature-local components
- Note the hooks and services needed, and where each lives
- One component per file; co-locate the feature under
  src/features/[leads]/
- Follow the structure conventions in AGENTS.md

Output the structure as a file tree with a one-line purpose per file.
Do NOT write implementation yet — wait for my approval of the structure.
```

Why it works: it forces a reviewable plan, enforces one-component-per-file, separates concerns, and ends with an explicit stop so you approve the shape before a single line is generated.

Prompt 2 — Generate a git commit message

Commits are the project's history. "update stuff" and "fix" tell future-you nothing. A consistent format keeps history searchable and lets tooling build changelogs.

▸ COPY-PASTE PROMPT

```
Generate a git commit message for the currently staged changes.

- Use Conventional Commits: type(scope): summary
  types: feat, fix, refactor, test, docs, chore, perf
- Summary line in imperative mood, max 72 characters
  ("add lead archive", not "added lead archive")
- Add a body explaining WHAT changed and WHY — not how
- Note any breaking change or follow-up explicitly
- One commit = one logical change. If the staged changes
  span multiple concerns, tell me to split them — do not
  write one message covering unrelated changes.
```

Prompt 3 — Add meaningful comments

Agents tend to over-comment (restating the obvious) or under-comment (no "why"). Good comments explain intent and non-obvious decisions — not what the next line literally does.

▸ COPY-PASTE PROMPT

```
Review [this file] and add comments ONLY where they add value.

- Explain WHY for non-obvious decisions — not WHAT the code does
- Add JSDoc/docstrings to exported functions and public APIs
- Flag edge-case assumptions and any TODO clearly
- Do NOT comment self-explanatory lines or restate the code
- Do not change any logic — comments only

Show the change as a diff so I can review what you added.
```

Make these team assets

Drop these into a shared `docs/prompts.md` or a Cursor/Antigravity workflow (a `/` slash command). The whole point is that nobody on the team writes these from scratch again.

TAKE THIS HOME

Don't rewrite the prompt every time. A reused prompt is a tested prompt.

05

MODULE 05 OF 11

Plan First, Document the Sprint

The highest-leverage habit in this handbook: **no non-trivial change starts without a written plan, and the plan lives in a file — not in chat history.**

Plan before you build

Ask the agent to produce a plan before it writes code. Review the plan as carefully as you would review the code — this is where wrong assumptions are cheapest to catch. Most agentic tools (Cursor's agent, Antigravity) produce a plan artifact by default — **do not auto-approve it to "get to the code faster."** Interrogating the plan is the point.

A plan file should contain

- **Goal** — what we're building and why
- **Approach** — the chosen solution in a few sentences
- **Files to touch** — the expected blast radius
- **Steps** — an ordered, reviewable task list
- **Risks & unknowns**, **test strategy**, and **out of scope**

Why markdown, why in the repo

Chat history is private to one person, unsearchable by the team, and gone when the thread closes. A markdown file in the repo is version-controlled, diffable, reviewable in a PR, searchable, and permanent. It also doubles as **reusable context** — point the agent straight at the plan file in future sessions.

Suggested `docs/` structure

```
# Everything below is committed to git
docs/
├── plans/        # one .md per feature, written before coding
├── sprints/      # one .md per sprint — the plan side
├── checkpoints/  # one .md per phase — the verification side (Module 06)
└── decisions/    # short ADR-style notes for architectural choices
```

A sprint note (`docs/sprints/sprint-14.md`)

```
## Sprint 14 — 2–13 Jun

### Goal
Ship the Lead module: add/edit, detail, and CSV import.

### Scope
- [x] Phase 5D — Add/Edit Lead + Lead Detail
- [ ] Phase 5E — Bulk Upload / Import     # in progress

### Decisions
- Dynamic fields sourced from Phase 5A config — see decisions/0009.
```

The sprint note is the **plan**: what we intend to do. Module 06 covers the **checkpoint**: proof of what actually got verified.

TAKE THIS HOME

If it isn't written in a file, it didn't happen.

06

MODULE 06 OF 11

Writing Tangible Sprint Phases

A sprint plan that lists features cannot be checked. A sprint plan that lists *what you should see on screen* can. This module is about writing phases tangible enough that anyone can verify them by hand — with no guesswork.

Why feature lists fail

"Add Lead works" — works how? when is it done? You cannot tell. Worse: an AI agent will happily report a feature "done" having stubbed it with local mock data. A vague feature list cannot catch that. A phase is not done when code exists — it is done when you can **click through it and watch every expected result happen for real.**

The format that works — three parts per phase

Part	What it is
Expected outcome	The functional goals — what the phase delivers.
Important	Explicit constraints and guardrails — what must *not* happen.
Manual outcome you should see	The exact clicks and the visible result. This is the acceptance test a human runs in the browser.

The gold-standard example — the format we follow

Phase 5D — Add/Edit Lead + Lead Detail

EXPECTED OUTCOME

- Add Lead works through backend
- Edit Lead works through backend
- Lead Detail opens a real backend lead
- Dynamic fields come from Phase 5A configuration
- Notes can be added / viewed
- Tasks / follow-ups can be added / viewed
- Status can be changed
- Lead can be archived
- **No fake / local lead records**

MANUAL OUTCOME YOU SHOULD SEE

- Click Add Lead → real form opens
- Save → lead appears in Lead Inbox
- Click row → backend Lead Detail opens
- Edit → save → refresh → changes remain
- Add note / task → visible after refresh
- Archive → lead disappears from default list

Phase 5E — Bulk Upload / Import / Mapping / Validation

EXPECTED OUTCOME

- CSV upload workflow foundation
- Field mapping screen
- Validation preview
- Invalid row detection
- Duplicate warning preview
- Commit only after user confirms
- Import history
- **No dirty direct import**

IMPORTANT

> CSV should **not** directly create leads without a preview.

MANUAL OUTCOME YOU SHOULD SEE

- Upload CSV
- Map CSV columns to configured lead fields
- Preview valid / invalid rows
- Fix mapping
- Commit valid rows
- Leads appear in Lead Inbox
- Invalid rows are reported clearly
- Import history is saved

Five rules for tangible phases

1. **Observable, not internal.** "Lead appears in Lead Inbox" — never "the createLead function works."
2. **State the negatives.** "No fake/local records", "No dirty direct import". Agents love to fake things with mock data — forbid it in writing.
3. **Manual outcome = a browser script.** Each line is a click or action ending in a result you can see.
4. **Binary items.** Each check passes or fails. There is no "mostly works."
5. **Small enough to check in minutes.** If verifying the phase takes an hour, the phase is too big — split it.

Close every phase with a checkpoint.md

When a phase is finished, run the Manual outcome list and record the result in `docs/checkpoints/checkpoint-<phase>.md`. This file is the **proof** the phase is done — and the honest starting point for the next phase.

Checkpoint template

```
# Checkpoint — Phase 5D: Add/Edit Lead + Lead Detail
Date: 2026-06-06   Status: PARTIAL

## Expected outcome — result
- [x] Add Lead works through backend
- [x] Edit Lead works through backend
- [ ] Lead can be archived  # archive endpoint 500s — issue #214

## Manual checks run
- [x] Click Add Lead -> real form opens
- [x] Save -> lead appears in Lead Inbox
- [ ] Archive -> lead disappears  # FAILS — still visible

## Carried to next phase
- Archive bug -> top of Phase 5E backlog

## Decisions / notes
- Confirmed dynamic fields come from Phase 5A config — no hardcoding.
```

Prompt to generate the checkpoint

▸ COPY-PASTE PROMPT

```
Phase [5D] is finished. Create docs/checkpoints/checkpoint-[5d].md.

- Restate every Expected outcome and Manual outcome item as a checkbox
- Mark [x] done / [ ] not done based ONLY on what we actually built
- Do NOT assume an item is done. If you cannot verify it, mark it
  unverified and say so explicitly — I will check it by hand
- List carried-over items, known issues (with links), and decisions
- Keep it factual. This file is the record we check against.
```

The trap

An agent asked to fill in a checkpoint will tick everything "done" by default. The checkpoint is only worth anything if a *human* runs the Manual outcome list and marks reality. The AI drafts the file; you confirm the checkboxes.

TAKE THIS HOME

A phase you can't verify by clicking through it isn't a plan — it's a hope.

07

MODULE 07 OF 11

Rules Files: Cursor & Antigravity

Our team works in **Cursor and Antigravity**. AI agents have no memory between sessions — without rules files you re-type your stack and conventions in every prompt, waste tokens, and get inconsistent code. Rules files are persistent project context the agent loads automatically.

Standardise on AGENTS.md as the single source of truth

Because we use two tools, the worst outcome is two diverging sets of rules. Avoid that: `AGENTS.md` **at the repo root is read by both Cursor and Antigravity** (and Claude Code). Make it the shared baseline — one file, one source of truth, committed to git. Tool-specific rule files then layer *on top*.

Cursor: the rules system

File / location	Status & use
`.cursorrules` (single file, root)	LEGACY Still works but deprecated — and **silently ignored in Agent mode**. Don't rely on it for agentic work.
`.cursor/rules/*.mdc` (directory of files)	CURRENT The modern format. Each `.mdc` file is markdown with a YAML frontmatter block for scoping. `.mdc` rules override `.cursorrules` on conflict.
`.cursorignore`	Keeps build output, large assets, and generated files out of the agent's context — saves tokens and noise.

Anatomy of a `.mdc` rule

```
---
description: API route conventions for the backend
globs: src/api/**/*.ts
alwaysApply: false
---
- All routes validate input with Zod before use.
- Return typed responses; never leak raw DB errors to the client.
- Use the asyncHandler wrapper for every handler.
```

The frontmatter sets one of **four activation modes**: **Always Apply** (every request), **Auto Attached** (when a file matching `globs` is touched), **Agent Requested** (the agent pulls it in via `description`), and **Manual** (invoked with `@rule-name`). Generate one from chat with `/Generate Cursor Rules`.

Antigravity: the rules system

In Antigravity, rules are passive markdown configured through **Agent Manager → ●●● → Customizations → Rules** (use *+Global* or *+Workspace*). Two files matter most:

- `AGENTS.md` (repo root) — the cross-tool foundation, shared with Cursor. Put the bulk of project rules here.
- `GEMINI.md` — Antigravity-specific rules; highest user priority, overrides `AGENTS.md` on conflict. Use only for Antigravity-only behaviour.

Antigravity also reads workspace rule files for organising rules by concern — configure these through the Customizations panel, which is the authoritative source within your installed version.

The minimum rule set for a repo

Don't write one 800-line mega-file — a large always-on rule is a "token tax" paid on every request, and a wall of text gets ignored. Keep it small and scoped. The practical target is **five to eight rule files**; more than ten usually means some can be merged.

File	Required?	Purpose
`AGENTS.md`	MANDATORY	Shared baseline: stack, structure, conventions, do/don'ts. Read by both tools.
`.cursor/rules/core.mdc`	MANDATORY	Always-apply essentials for Cursor — keep it short (~200 words).
`.cursor/rules/frontend.mdc`	Recommended	Auto-attached on `src/components/**`.
`.cursor/rules/api.mdc`	Recommended	Auto-attached on `src/api/**`.
`.cursor/rules/testing.mdc`	Recommended	Vitest / Playwright expectations.
`.cursor/rules/security.mdc`	Recommended	No hardcoded secrets, input validation, safe defaults.
`.cursorignore`	Recommended	Excludes build output and noise from agent context.
`GEMINI.md`	Optional	Only if you need Antigravity-only behaviour.

Put in a rules file

- Tech stack *with versions*
- Folder structure & naming conventions
- "Plan before coding; keep diffs small"
- Testing & security requirements
- Patterns to reuse, libraries to avoid

Never put in a rules file

- Secrets, API keys, tokens, credential URLs
- Large dumps of code or whole files
- Anything that changes per task
- Vague aspirations ("write good code")

Starter `AGENTS.md` skeleton

```
# Project: Acme CRM

## Stack
- Next.js 15 (App Router), TypeScript strict, Tailwind
- Node API routes, PostgreSQL via Prisma
- Vitest + React Testing Library, Playwright for E2E

## Structure
- Feature code in src/features/<feature>/ — one feature per folder
- Shared UI in src/components/, never business logic there
- One component per file

## Workflow rules
- Produce a plan before any multi-file change; wait for approval.
- Keep changes small and single-purpose. Every feature ships with tests.

## Hard rules
- Never hardcode secrets — read from process.env only. Never commit .env.
- Never use fake/local/mock data to fake a feature being "done".
- Validate all external input with Zod. Match existing patterns.
```

Maintenance habit

When the agent makes the same mistake three times, that's a missing rule — add one focused rule. When a rule hasn't been needed in weeks, delete it. Commit the whole `.cursor/rules/` folder and `AGENTS.md` so the team stays in sync.

TAKE THIS HOME

Write the rule once. Stop re-explaining your project in every single prompt.

08

MODULE 08 OF 11

The Minimum File Set

Before any real work starts — scaffolding from scratch or adopting a repo — these files should exist. They are cheap to create and expensive to retrofit. A repo missing them is not "early," it is incomplete.

File	Committed?	Purpose
`README.md`	YES	What the project is, prerequisites, how to install and run it locally.
`.gitignore`	YES	Must exist *before the first commit*. Excludes secrets, dependencies, build output.
`.env.example`	YES	Every required environment variable, with placeholder values.
`.env`	NEVER	Real secrets and local config. Local only, git-ignored.
`AGENTS.md`	YES	The shared AI rules baseline for Cursor + Antigravity (Module 07).
`.cursor/rules/*.mdc`	YES	Scoped Cursor rules — commit so the team shares them.
`.cursorignore`	YES	Keeps noise out of the agent's context window.
`package.json` + lockfile	YES	Dependencies and scripts. **Commit the lockfile** — it pins exact versions.
`docs/`	YES	Plans, sprint notes, checkpoints, decisions (Modules 05 & 06).
Lint / format config	YES	Shared code style, enforced not debated. Run in CI.
Test config `vitest / playwright`	YES	So tests run identically on every machine and in CI.
`.github/workflows/ci.yml`	YES	CI: lint, test, build on every PR. A red pipeline blocks merge.

The non-negotiable five

If you do nothing else on day one, a repo must have: `README.md`, `.gitignore`, `.env.example`, `AGENTS.md`, and a committed lockfile. Everything else can follow within the first sprint — these five cannot.

When the agent scaffolds a project for you

AI tools often skip `.gitignore`, `.env.example`, and rules files when generating a project. Always check the file list against this table before the first commit.

TAKE THIS HOME

Five files on day one. Retrofitting them after a breach costs ten times more.

09

MODULE 09 OF 11

.gitignore & .env Discipline

Leaked secrets are one of the most common and most damaging real-world security failures — and they are almost always preventable with two small files.

The three files and their jobs

File	In git?	Job
`.gitignore`	COMMITTED	Tells git what to never track — secrets, dependencies, build output, OS/ editor junk.
`.env`	IGNORED	Holds the real secrets and local config: API keys, database URLs, tokens.
`.env.example`	COMMITTED	The same variable names with placeholder values — so teammates and agents know what `.env` must contain.

Why this matters more than it looks

Git keeps history. Once a secret is committed, it is preserved in every clone of the repo's history — **deleting the file in a later commit does not remove it.** The only real fix after a leak is to **rotate the secret** (invalidate the old key, issue a new one) and then purge it from history. Preventing the commit costs one line in `.gitignore`.

> **The AI-specific risk**
>
> Coding agents will cheerfully hardcode an API key into a source file, or write a script that prints `.env` into a committed file. They optimise for "make it run," not "keep it secret." State the rule explicitly in `AGENTS.md` — and still check the diff for it.

Baseline `.gitignore` (Node / web)

```
# secrets & environment
.env
.env.*
!.env.example          # keep the example, ignore the rest

# dependencies & build output
node_modules/
dist/
build/
.next/
coverage/

# logs & OS / editor noise
*.log
.DS_Store
.idea/
```

Matching `.env.example`

```
# Copy to .env and fill in real values. Never commit .env.
DATABASE_URL=postgresql://user:password@localhost:5432/appdb
JWT_SECRET=replace-with-a-long-random-string
STRIPE_SECRET_KEY=sk_test_xxxxxxxxxxxxxxx
```

> **If a secret was already committed**
>
> **1)** Rotate the secret immediately — assume it is compromised. **2)** Then purge it from git history. **3)** Add it to `.gitignore`. Rotation is the step people skip — it is the one that actually closes the hole.

TAKE THIS HOME

A committed secret is a leaked secret. There is no "delete it later."

10

MODULE 10 OF 11

QA Automation: Vitest & Playwright

AI-generated code looks correct. That is exactly the problem — it looks correct whether or not it is. Automated tests are how we verify at speed instead of hoping.

Why tests matter *more* with AI code

When you write code by hand, you build a mental model of it as you go. When an agent writes it, that model doesn't exist until you read the code — and reading catches less than you think. Tests are the independent check: they encode the expected behaviour and fail loudly when the agent's confident-but-wrong output breaks it.

Vitest — fast unit & component testing

A Vite-native test runner with a Jest-compatible API. It tests the small pieces in isolation — functions, utilities, services, and (with React Testing Library) component logic. It is fast, so you run it constantly and keep a tight feedback loop. It answers: **"is this individual unit correct?"**

Playwright — end-to-end testing

Playwright drives a real browser (Chromium, Firefox, WebKit) and exercises real user journeys: log in, fill a form, create a record, navigate. It catches the integration failures unit tests miss. It answers: **"does the whole flow actually work for a user?"**

	Vitest	Playwright
Scope	One unit / component in isolation	A full user flow across the running app
Speed	Very fast — run on every save	Slower — run before merge and in CI
Count	Many — the bulk of the suite	Fewer — the critical journeys
Catches	Logic errors, edge cases, regressions	Broken integrations, routing, real-world breakage

It is not a choice between them — you want both. Many cheap, fast unit tests for correctness; a smaller set of end-to-end tests for the journeys that must never break.

How we work with tests

- **Every feature ships with tests.** A PR with new behaviour and no tests is incomplete.
- **Tests run in CI on every PR.** A red pipeline blocks merge — no exceptions.

- **Let the agent draft tests, then review them like code.** AI writes good test scaffolding — it also writes tautological tests that assert the implementation, or that encode an existing bug. Read them.
- **Write the test against the requirement** — the Manual outcome list from Module 06 — not against whatever the code currently does.

The trap to watch for

An agent asked to "make the tests pass" can do so by weakening the tests. An agent asked to "write tests for this code" can write tests that pass on buggy code because they were derived from that same buggy code. Tests only protect you if they encode the *intended* behaviour — which a human has to define and check.

TAKE THIS HOME

"It runs" is not "it works." A green pipeline is the only proof that ships.

MODULE 11 OF 11

11 The Operating Standard

The whole handbook compressed into the standard we hold each other to. If a piece of work doesn't meet these, it isn't done — regardless of how fast the agent produced it.

Before you code

- A written **plan exists in a file** and has been reviewed.
- The phase is written as a **tangible spec** — Expected outcome + Manual outcome you can verify by hand.
- Repo has its **minimum file set** and current **rules files**.

While you code

- **One task per chat, one concern per PR.** Small, reviewable diffs.
- Prompts are **specific** — reuse the Module 04 playbook.
- Start a **fresh chat** when the task changes.
- **Read every line** the agent produces. If you can't explain it, it doesn't merge.
- **No secrets in code, ever.** No fake/local data faking a feature "done."

Before it merges

- The feature has **tests** you have read — checking intent, not just implementation.
- **CI is green**: lint, unit, E2E, build.
- The **checkpoint.md** is written and its boxes reflect a real manual run.
- The diff is small enough for a reviewer to genuinely review.

Hold onto

- The AI drafts; you decide.
- Plan first, build second.
- Precision up front saves tokens.
- Write it down or it's lost.
- Verify — don't trust.

Let go of

- "It runs, so it's done."
- "The AI said it works."
- "I'll document it later."
- "Looks fine, merge it."
- One huge prompt for everything.

TAKE THIS HOME

Speed and standards. The day it becomes speed instead of standards, we've lost the plot.

★ Vibe Coding Cheat Sheet

Print this. Pin it next to your monitor. The whole handbook at a glance.

GOOD PROMPT FORMULA

- **Goal** — one outcome, one sentence
- **Context** — exact file(s) + pattern to follow
- **Constraints** — what must NOT change
- **Acceptance** — how "done" is verified
- **Out of scope** — what to leave alone

COMPONENT STRUCTURE PROMPT

- "Before any code, propose a component-wise file tree."
- One component per file; container vs presentational
- Co-locate under `src/features/<name>/`
- End with: "wait for my approval — do not implement yet"

COMMIT MESSAGE PROMPT

- Conventional Commits: `type(scope): summary`
- Imperative mood, ≤ 72 chars; body = what + why
- One commit = one logical change — else split

SAVE TOKENS

- One task → one chat. New task → new chat.
- Name exact files; don't make the agent search.
- Stack & conventions live in rules, not prompts.
- Don't paste files the agent can index. Scope small.

TANGIBLE PHASE FORMAT

- **Expected outcome** — functional goals
- **Important** — constraints / "no fake data"
- **Manual outcome** — exact clicks → visible result
- Observable, not internal. Binary. Checkable in minutes.

CHECKPOINT.MD

- One per phase → `docs/checkpoints/`
- Restate outcomes as checkboxes; mark real pass/fail
- A *human* ticks the boxes — not the AI
- Record carryover, issues, decisions

RULES FILES MAP

- `AGENTS.md` — root, shared by Cursor + Antigravity
- `.cursor/rules/*.mdc` — scoped, 5–8 files
- `.cursorrules` — legacy, ignored in Agent mode
- `GEMINI.md` — Antigravity-only overrides

MINIMUM REPO FILES

- `README` · `.gitignore` · `.env.example`
- `AGENTS.md` · `.cursor/rules/` · lockfile
- `docs/` · lint/test config · CI workflow
- **Non-negotiable 5:** README, .gitignore, .env.example, AGENTS.md, lockfile

.GITIGNORE MUST-HAVES

- `.env` & `.env.*` — keep `.env.example`
- `node_modules/` · `dist/` · `.next/`
- `coverage/` · `*.log` · `.DS_Store`
- Must exist **before** the first commit.

PRE-MERGE CHECKLIST

- ☐ Read every line of the diff
- ☐ Tests exist and check intent
- ☐ CI green: lint + test + build
- ☐ No secrets in code
- ☐ checkpoint.md reflects a real manual run

TESTING QUICK MAP

- **Vitest** — units, fast, many. "Unit correct?"
- **Playwright** — real-browser E2E, fewer. "Flow works?"
- Use both. Feature ships with tests. CI runs them.
- Review AI tests — they can encode the bug.

NEVER DO

- Merge code you haven't read
- Commit `.env` or hardcode a secret
- Start building with no plan
- Trust "the AI said it works"
- Fake a feature with local/mock data
- Ship a feature with no tests
- Cram many features into one prompt
- Auto-approve the agent's plan to "go faster"

THE WHOLE HANDBOOK IN ONE LINE

The AI drafts — you are the engineer of record. Plan first, prompt precisely, keep diffs small, verify with tests, write it down.